Design of a Low-Latency RDMA-Based Publish/Subscribe Framework for Inter-Machine GPU Communications*

Ren-Mao Jiang, Yu-Te Ku, and Shih-Hao Hung

National Taiwan University, Taiwan
{r12944042,hunghsh}@csie.ntu.edu.tw, d08946006@ntu.edu.tw

Abstract

Modern GPU-centric applications increasingly execute as multi-stage pipelines in which one stage produces large payloads directly in GPU memory and multiple downstream stages must consume the same data with low latency. In conventional Publish/Subscribe models, messages are disseminated from a publisher to all subscribers registered to a specific topic. However, this paradigm is rarely applied to GPU-centric applications. This paper presents Remote GPU-Aware Publish/Subscribe (RGAPS), a low-latency framework for cross-machine GPU communication. RGAPS extends single-host GPU-aware pub/sub architecture to a two-host distributed setting by combining shared CUDA-RDMA pinned memory for intra-node aggregation, RDMA Write with Immediate for CPU-bypassing inter-node transport, and a shared GPU memory pool for one-time materialization and local fan-out on the subscriber machine. The design preserves the "publish once, fan-out many" principle across machines while avoiding per-subscriber payload duplication. Experiments on a real two-node platform with RTX 3090 GPUs, ConnectX-7 RNICs, PCIe Gen4, and InfiniBand NDR show that RGAPS reduces end-to-end latency by roughly 6×–10× relative to conventional CPU-centric baselines and keeps local fan-out cost nearly insensitive to the number of subscribers. These results demonstrate that a carefully integrated RDMA-based data plane can substantially improve the subscriber scalability and latency of inter-machine GPU publish/subscribe communication.

1 Introduction

The publish/subscribe (pub/sub) pattern is a widely used communication abstraction that decouples publishers (producers) from subscribers (consumers) in space, time, and synchronization. Publishers send data to topics rather than to specific receivers, and the middleware is responsible for routing and dissemination. This decoupling is attractive in large-scale systems because it simplifies software composition and supports many-to-many communication.

Different pub/sub systems prioritize different goals. Apache Kafka [12] emphasizes high throughput, durable storage, and replay through a distributed log architecture. ROS 2 relies on DDS to support modular and real-time robotic pipelines. Eclipse Iceoryx [13, 15] focuses on deterministic intra-machine inter-process communication and achieves zero-copy delivery for host-resident data through shared memory. Eclipse Zenoh [14, 16] targets geographically distributed and heterogeneous deployments by combining lightweight pub/sub with query functionality. These systems provide valuable abstractions, but their dominant data path remains host-centric.

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 49, July 8-10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

This assumption falls short of supporting modern GPU-driven workloads. In many real-time and high-throughput pipelines, producers generate results directly in GPU memory, and downstream stages also execute on GPUs. Typical payloads include tensors, feature maps, image batches, point clouds, embeddings, or intermediate outputs of deep learning models. When conventional pub/sub middleware is placed between such stages, GPU-produced payloads are staged through host memory, transmitted by CPU-centric communication paths, and often copied back into GPU memory separately for each subscriber. As message sizes grow from megabytes to gigabytes, these redundant data movements become a major contributor to end-to-end latency and can amplify contention on PCIe and memory subsystems.

A prior work on single-host GPU-aware publish/subscribe [3] addresses this problem by treating GPU memory as a first-class data plane. Instead of distributing payload copies, single-host GPU-aware publish/subscribe distributes lightweight metadata that allows local subscribers to access a shared GPU-resident buffer. This design removes per-subscriber payload duplication and achieves stable low latency for intra-node fan-out. However, it is fundamentally limited by host locality: publishers and subscribers must reside on the same machine, and each topic consumes GPU memory capacity in a single-host shared pool. As workload diversity and topic count increase, GPU memory pressure and resource placement rigidity become important limitations.

This paper presents Remote GPU-Aware Publish/Subscribe (RGAPS), a low-latency RDMA-based pub/sub framework for inter-machine GPU communication. RGAPS extends the main idea of single-host GPU-aware publish/subscribe from single-host GPU fan-out to a distributed two-host setting. The framework stages outgoing payloads through a GPU-accessible RDMA-registered pinned host memory region on the publisher machine, transfers them with RDMA Write with Immediate into a GPU-accessible RDMA-registered receive-side ring buffer, and materializes them exactly once into a shared GPU memory pool on the subscriber machine. Multiple local subscribers then access the same GPU-resident payload without additional per-subscriber copies.

The main contributions of this paper are as follows:

1. We design a distributed GPU-aware publish/subscribe architecture that preserves the "publish once, fan-out many" principle across machines by combining shared CUDA-RDMA pinned memory, one-sided RDMA transport, and shared-GPU local fan-out.
2. We introduce an RDMA ring-buffer design together with an RDMA Write with Immediate receive path that removes costly receive-buffer sanitization from the critical path while maintaining safe variable-sized message streaming, effectively eliminating the performance bottlenecks inherent in the conventional, naive socket-based TCP/IP approach.
3. We evaluate the framework on a real two-node GPU-RDMA platform and show that RGAPS substantially reduces both path-level and end-to-end latency relative to pageable-memory, socket-based, and CPU-centric fan-out baselines.

2 Background and Related Work

2.1 Single-Host GPU-Aware Publish/Subscribe

Single-host GPU-aware publish/subscribe [3] is the direct conceptual precursor to RGAPS. As illustrated in Figure 1, this design decouples data placement from control signaling by storing

large payloads in shared GPU memory and using shared host memory only for metadata and notification. A publisher writes a payload directly into a shared GPU memory pool, stores the payload size and offset in shared host memory, and sends a lightweight notification to subscribers. Each subscriber reads the metadata and directly accesses the payload from the shared GPU pool. This design eliminates redundant host-device transfers and achieves zero-copy local fan-out.

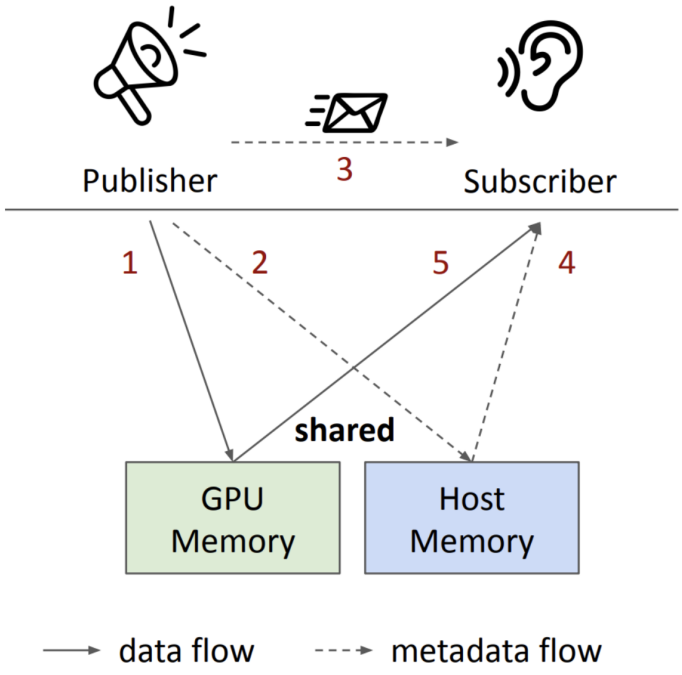


Figure 1: The architectural workflow of single-host GPU-aware publish/subscribe.

The main limitation of the design is that it operates only within one machine. All participants must share the same host, and the per-topic GPU memory pool must be provisioned within local GPU memory. As the number of topics and service stages increases, both GPU memory pressure and compute resource contention, coupled with resource placement constraints, become problematic.

2.2 Pinned Host Memory

Pinned host memory is important for efficient GPU–host transfer. Unlike pageable memory, which requires an extra CPU-mediated copy through a driver-managed pinned staging buffer in kernel space, pinned memory enables a direct DMA path between device memory and user space host memory [8, 9]. In practice, this can reduce device-to-host transfer latency by around 1.8×–2.0× on sufficiently provisioned PCIe links, as summarized in Figure 2.

Pinned memory also provides the stable physical residency required for RDMA memory registration. This makes it a natural staging substrate for a GPU–network–GPU pipeline in environments where the RNIC cannot directly access GPU memory.

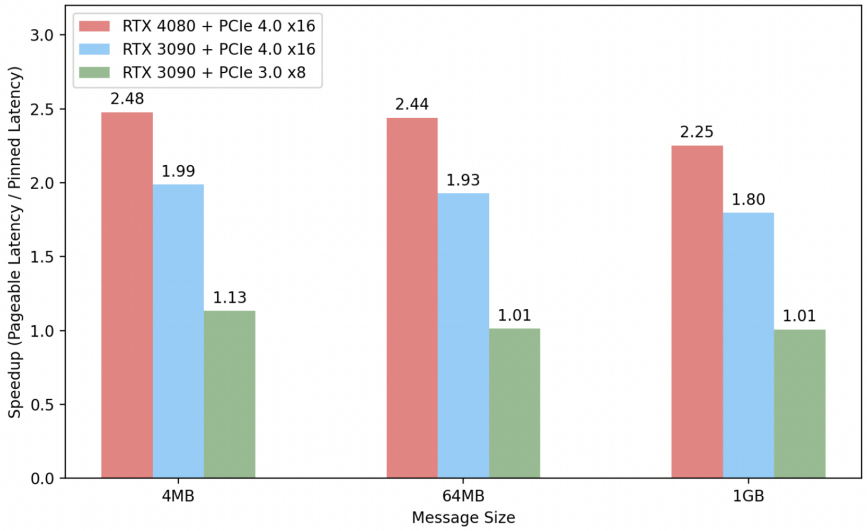


Figure 2: Impact of PCIe bandwidth and GPU micro-architecture on pinned-memory speedup.

2.3 RDMA and Distributed Messaging

RDMA reduces software overhead by enabling kernel-bypass and zero-copy networking. In one-sided operations, an initiator can directly write to or read from a remote registered memory region without involving both local and remote CPUs in the data path [7, 4, 2]. These properties make RDMA especially attractive for latency-sensitive communication involving large payloads.

FaRM [1] demonstrated the use of RDMA ring buffers for lightweight distributed messaging. Its design is effective for CPU-resident data, but its validity test depends on memory-header polling and buffer zeroing, which can become expensive for large payloads. KafkaDirect [11] accelerates Apache Kafka with RDMA, but its focus is broker-managed log semantics rather than direct dissemination of GPU-resident payloads for immediate downstream execution.

RGAPS differs from these systems in two ways. First, it targets GPU-resident payloads rather than host-centric messages. Second, it combines inter-node RDMA transport with a receive-side shared GPU fan-out design so that the same payload can be reused by multiple local GPU consumers after a single materialization step.

3 RGAPS Design Overview

As shown in Figure 3, RGAPS is organized around the idea that each message should be copied as few times as possible while still supporting cross-machine dissemination and one-to-many fan-out. The end-to-end data plane is divided into three stages:

1. **Intra-node aggregation** on the publisher machine.
2. **Inter-node transport** across the network.
3. **Local dissemination** on the subscriber machine.

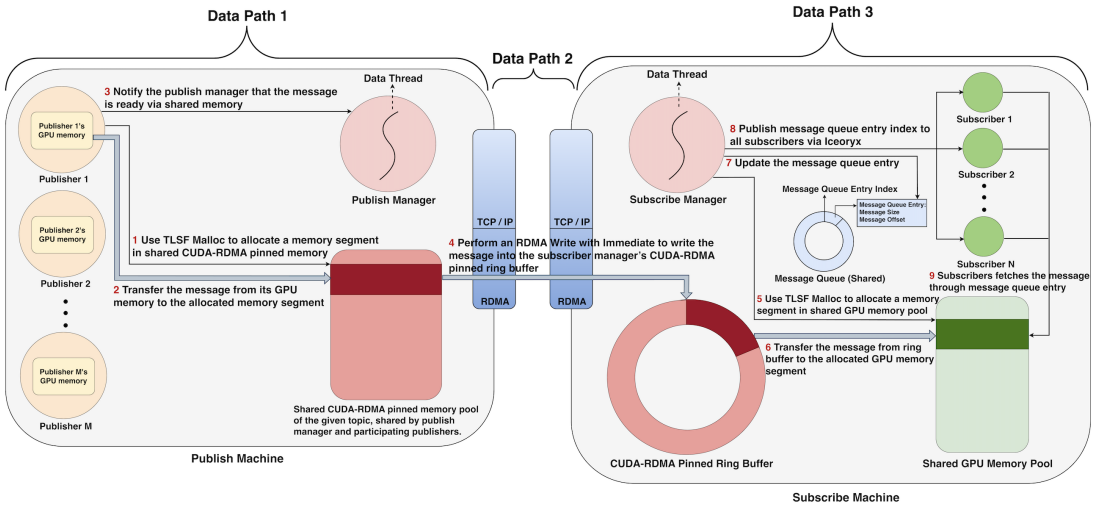


Figure 3: Architectural overview of the RGAPS data plane.

The control plane, shown in Figure 4, is separated from the high-speed data plane. Standard TCP/IP and Unix Domain Sockets are used for topic setup, resource discovery, and publisher/subscriber registration, while RDMA is reserved for a part of performance-critical data plane.

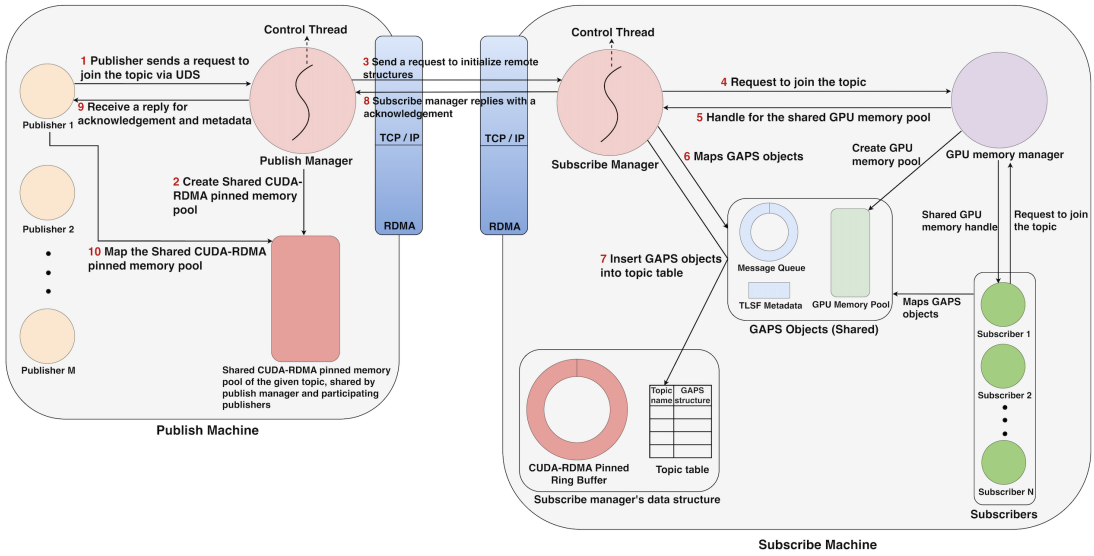


Figure 4: Control-plane architecture and initialization process.

The overall architecture is physically bifurcated into two distinct nodes: a publisher machine and a subscriber machine. For any given topic, all publishers are consolidated on the publisher machine, while all subscribers reside on the subscriber machine. To mediate the continuous stream of messages between these two nodes, each machine hosts a dedicated intermediary

process—specifically, a publish manager and a subscribe manager.

Within these manager processes, a control thread is responsible for topic initialization and the establishment of per-topic metadata. Additionally, a dedicated data thread manages the local aggregation and dissemination of messages, as well as the continuous inter-node transmission of payloads.

In the publisher machine, the publish manager maintains a per-topic shared CUDA-RDMA pinned memory pool. This pool serves to receive payloads from local publishers and acts as the origin buffer for RDMA-based payload transfers. Correspondingly, the subscribe manager maintains a global CUDA-RDMA pinned ring buffer in the subscriber machine. This buffer functions as the destination endpoint for incoming RDMA transfers from the publish manager, from which the payloads are subsequently disseminated to all local subscribers registered to the designated topic.

The overall data plane is logically partitioned into three distinct segments: Data Path 1, Data Path 2, and Data Path 3. Specifically, Data Path 1 encompasses the set of all intra-node data flows on the publisher machine. Data Path 2 constitutes the inter-node data flow responsible for message transmission between the publisher and subscriber machines. Finally, Data Path 3 comprises the set of all intra-node data flows on the subscriber machine.

3.1 Data Path 1: Intra-Node Aggregation

As illustrated in Figure 5, RGAPS implements Data Path 1 (Pinned-SHM) on the publisher machine by creating a shared CUDA-RDMA pinned memory pool. The figure contrasts this proposed design with the baseline approach, Data Path 1 (Pageable-UDS), which relies on pageable memory and Unix domain sockets. In Data Path 1 (Pinned-SHM), the memory region is shared across local publisher processes, registered for RDMA, and accessible to the GPU. Publishers allocate segments within this pool with a TLSF-based allocator [5, 10]. After allocation, a publisher performs a single *cudaMemcpy* from its GPU memory into the shared pinned segment and writes only lightweight metadata to notify the publish manager via shared memory.

This design removes the intermediate kernel-space memory copies inherent in standard device-to-host transfers, alongside the heavy IPC cost of forwarding the full payload through a Unix Domain Socket, and ensures that the payload is written to host memory only once before network transmission.

3.2 Data Path 2: Inter-Node Transport

The publish manager transfers messages from the shared pinned pool to the subscriber machine through RDMA Write with Immediate. The destination is a pre-allocated pinned ring buffer owned by the subscribe manager. One-sided RDMA Write moves the message directly into that buffer, and the immediate data field delivers the message size through the receiver's completion queue.

The standard socket-based approach routes cross-node data streams directly into the subscribe manager's pageable host memory. However, relying on the OS network stack introduces some major software-bound bottlenecks: Processing packets demands intense CPU involvement, forcing expensive transitions between user and kernel spaces. Furthermore, messages suffer from

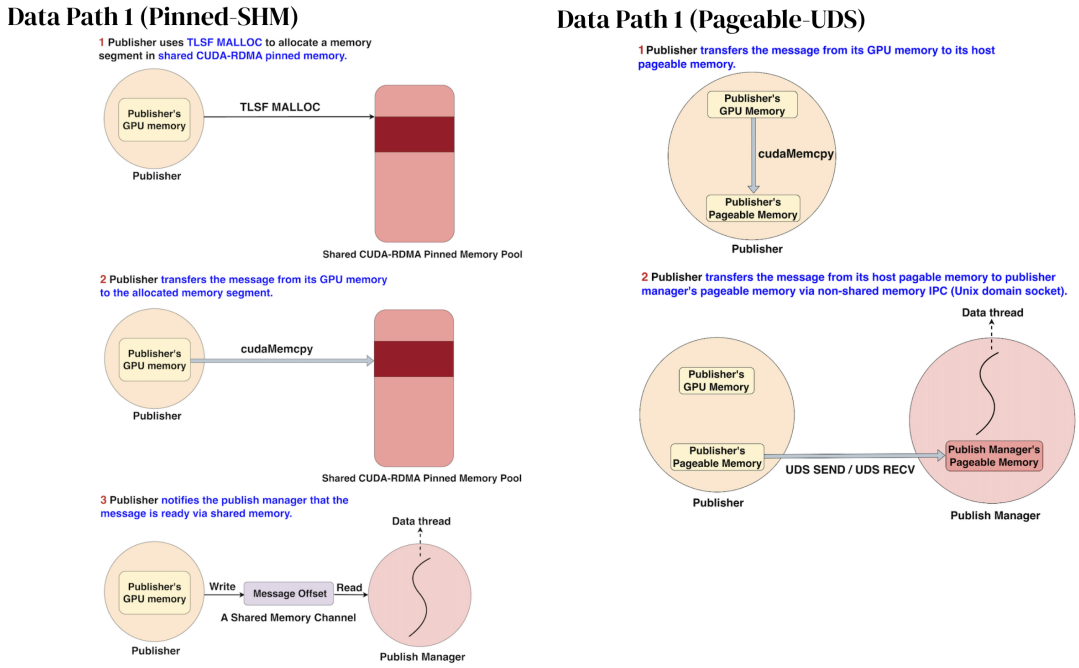


Figure 5: Comparison of Data Path 1: Pinned-SHM versus Pageable-UDS.

redundant copying between user and kernel spaces, potentially triggering additional context switches.

In contrast, this RDMA transport path avoids both local and remote CPU intervention, eliminates kernel networking overhead, and keeps the message in GPU-accessible pinned memory once it arrives on the subscriber machine.

3.3 Data Path 3: Local Dissemination

The final stage of the pipeline, Data Path 3, governs the local distribution of messages from the subscribe manager to multiple distinct subscribers running on the destination node. This phase is critical for determining the system's fan-out scalability regarding the number of subscribers.

To eliminate redundant memory transfers, the proposed design, denoted as Data Path 3 (Pinned-Shared), completely decouples payload placement from notification logic. After a message lands in the receive-side pinned ring buffer, the subscribe manager utilizes the TLSF algorithm to allocate a single memory segment within a shared GPU memory pool. It then executes a single host-to-device transfer (*cudaMemcpy*) to move the message directly into this allocated shared GPU segment. Instead of broadcasting the full message, the manager utilizes Iceoryx solely to publish lightweight metadata—specifically, the per-topic shared message queue entry index.

Upon receiving this index, each subscriber independently computes and resolves the correct message address within the shared GPU pool using the message base address and offset recorded in the designated shared message queue entry. Since the GPU memory is pre-mapped into every subscriber's address space, they can immediately access the GPU-resident payload without

performing any additional memory copies.

In a conventional CPU-centric baseline, denoted as Data Path 3 (Pageable-Iceoryx), the subscribe manager publishes the full payload via Iceoryx to the host pageable memory of all subscribers. Consequently, each individual subscriber must perform its own *cudaMemcpy* to transfer the data into its private GPU memory. This naive approach mandates N redundant host-to-device transfers for N subscribers, inevitably causing linear bandwidth saturation on the PCIe bus. By contrast, RGAPS performs exactly one host-to-device transfer per message rather than one per subscriber, drastically minimizing PCIe bandwidth consumption and ensuring stable performance even with a high density of subscribers.

4 RDMA Ring Buffer and Receive-Side Optimization

The receive-side RDMA ring buffer is a central component of RGAPS. It supports continuous streaming of variable-sized messages from the publishing node while avoiding overwrite of unconsumed data. The design uses three pointers: a local write cursor (*Tail*) maintained by the publish manager, a true read cursor (*Head*) maintained by the subscribe manager, and a cached lower-bound estimate of the remote head (*Delayed Head*) maintained on the publishing side.

Before issuing an RDMA Write, the publish manager checks whether *Tail + message size* would overrun the current *Delayed Head*. Because *Delayed Head* is a conservative view of the remote consumption progress, this local test guarantees safe writes without requiring a remote read for every message.

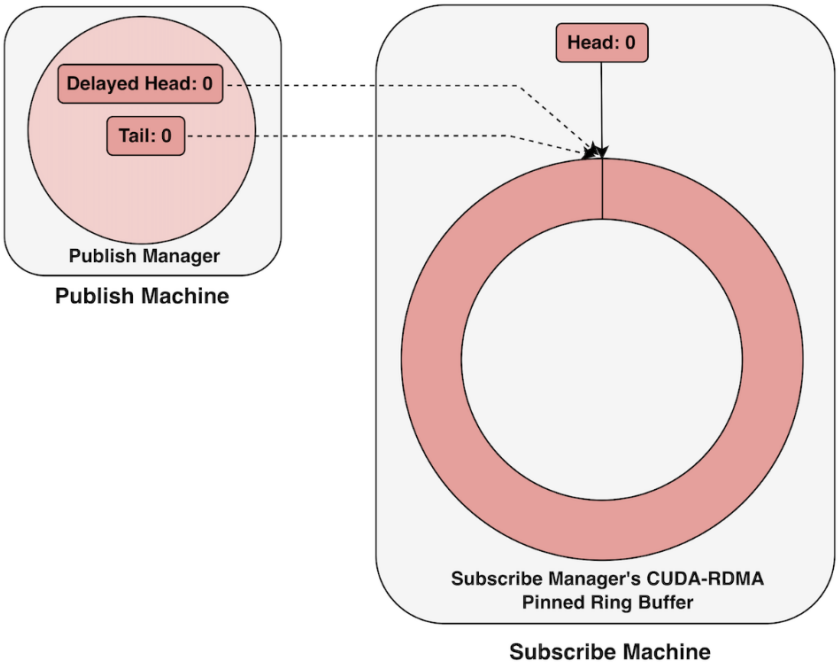


Figure 6: Initial state of the RDMA ring buffer.

Figure 6 shows the initial state of the ring buffer. As messages are written, the publish manager advances *Tail*. The subscribe manager consumes messages by advancing *Head*. To reduce control traffic, the subscribe manager updates *Delayed Head* only after a configurable watermark of consumed bytes is reached. Figures 7, 8, and 9 illustrate the write, consume, and delayed-head update phases, respectively.

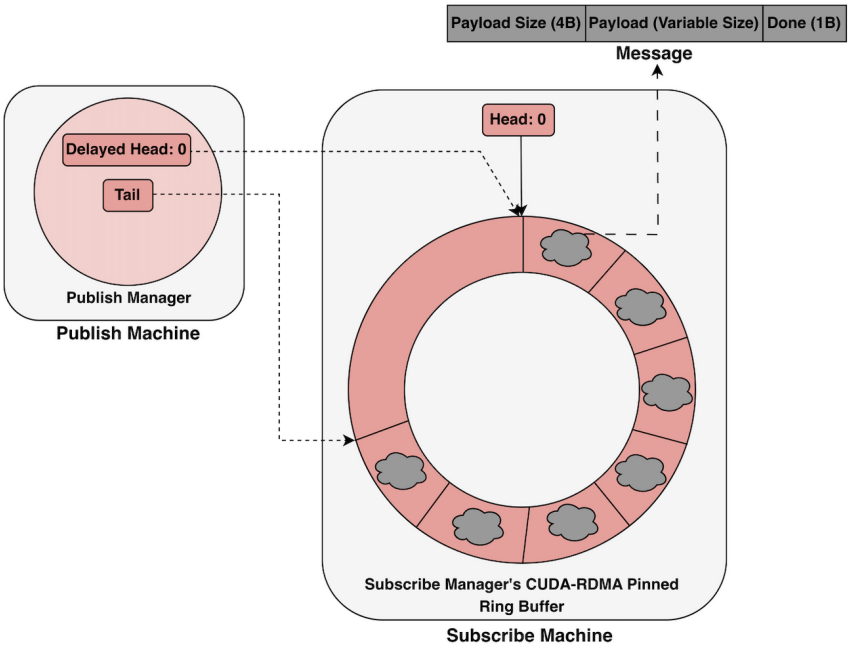


Figure 7: Multiple messages written to the RDMA ring buffer.

An important design issue is how the receiver detects message arrival. In a naive memory-polling design, the subscriber must look for a non-zero header, fetch and process the message, and then zero out consumed messages to prevent stale garbage values from being mistaken as fresh messages after wrap-around. We quantify this memory zero-out overhead in Figure 10; for 1 GB messages, local zeroing costs about 91.4% of the latency of an RDMA write of the same size.

To remove this overhead, RGAPS uses RDMA Write with Immediate. Instead of polling message headers in memory, the subscribe manager polls the completion queue. The payload size is carried in the immediate data field and is therefore received as completion metadata rather than as part of the payload header. This design eliminates the need for receive-buffer sanitization on the critical path and makes the transport cost depend primarily on network and DMA performance rather than on extra CPU-side memory maintenance.

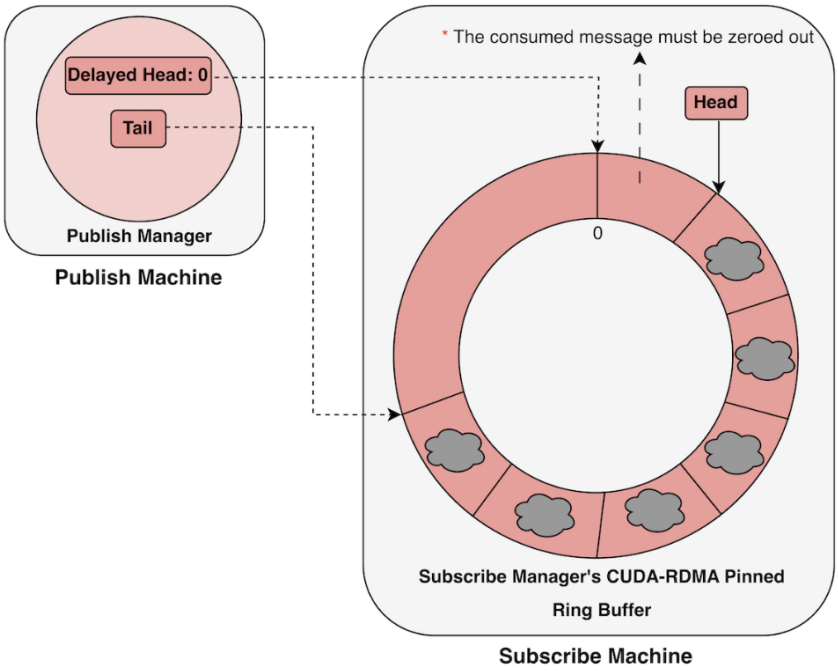


Figure 8: Consumption of messages from the RDMA ring buffer.

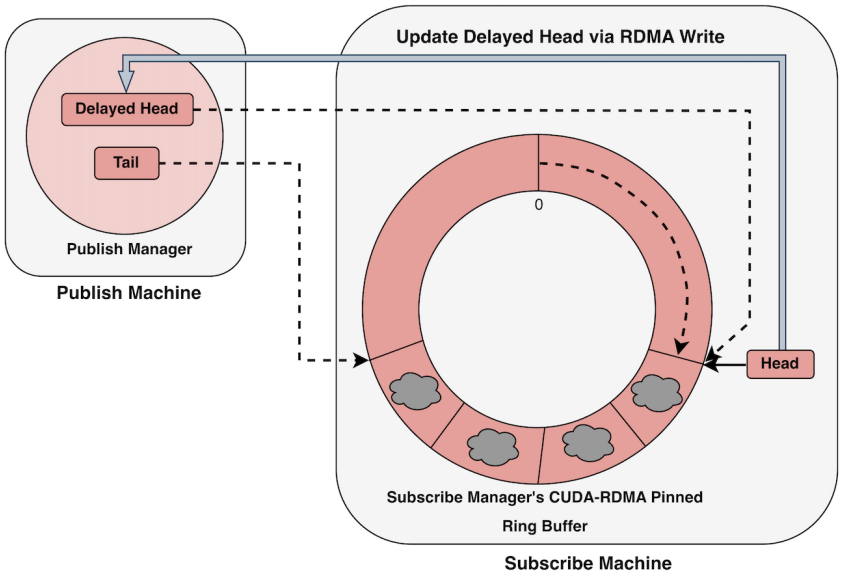


Figure 9: Updating the publish manager's delayed head.

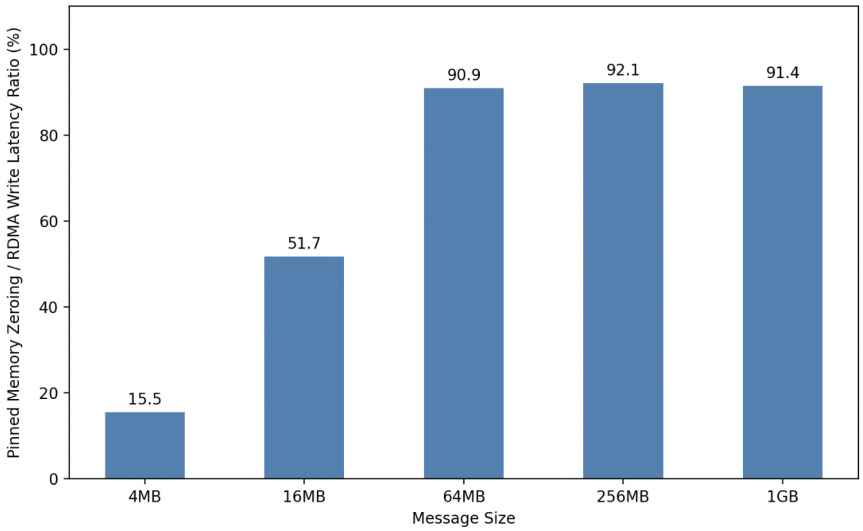


Figure 10: Overhead of pinned-memory zero-out relative to RDMA Write latency.

5 Evaluation

5.1 Experimental Setup

The evaluation platform consists of two symmetric workstations, one acting as the publish machine and the other as the subscribe machine. Table 1 summarizes the hardware and software configuration. Each machine contains an AMD Ryzen Threadripper 3970X CPU with 32 cores and 64 threads, 128 GB of DDR4-2666 memory, an NVIDIA GeForce RTX 3090 GPU with 24 GB of device memory, and an NVIDIA ConnectX-7 RNIC. The GPU and RNIC are connected through independent PCIe Gen4 x16 interfaces. The two machines are linked by InfiniBand NDR. Ubuntu 22.04, CUDA 12.7, and Iceoryx 2.0.6 were used.

We compare RGAPS against three baseline paths derived from conventional CPU-centric communication: pageable-memory plus Unix-domain-socket IPC for local aggregation, TCP/IP for inter-node transmission, and CPU-oriented local fan-out that requires separate host-to-device copies per subscriber.

5.2 Data Path 1 Results

Data Path 1 measures the transfer of a payload from publisher GPU memory to host memory on the publisher machine. RGAPS uses the proposed Pinned-SHM design, while the baseline uses pageable memory plus Unix Domain Sockets.

Regarding the device-to-host memory transfer latency, for a 4 MB message, RGAPS reduced latency from 451 μs to 224 μs. For 64 MB, it reduced latency from 5,709 μs to 2,906 μs. For a 1 GB message, latency fell from 85,806 μs to 47,905 μs. These gains are shown in Figure 11 and arise from eliminating the double-copy behavior of pageable memory.

Table 2 and Table 3 provide a granular total latency breakdown of Data Path 1 to contrast the

Table 1: Hardware and software configurations of the publish and subscribe machines.

	Publish Machine	**Subscribe Machine**
Hardware		
CPU	AMD Ryzen Threadripper 3970X (32 cores / 64 threads)	AMD Ryzen Threadripper 3970X (32 cores / 64 threads)
GPU	NVIDIA GeForce RTX 3090 (10496 CUDA cores, 24GB), PCIe Gen4 x16	NVIDIA GeForce RTX 3090 (10496 CUDA cores, 24GB), PCIe Gen4 x16
RAM	128 GB DDR4-2666 (4 × 32 GB)	128 GB DDR4-2666 (4 × 32 GB)
Adapter	NVIDIA ConnectX-7 — PCIe Gen4 x16	NVIDIA ConnectX-7 — PCIe Gen4 x16
Interconnect	InfiniBand NDR 400 Gb/s	
Software		
OS	Ubuntu 22.04 (kernel: 6.8.0-51-generic)	Ubuntu 22.04 (kernel: 6.8.0-87-generic)
Compiler	nvcc v11.5, gcc 11.4.0, g++ 11.4.0	nvcc v11.5, gcc 11.4.0, g++ 11.4.0
CUDA	12.7	12.7
Iceoryx	2.0.6	2.0.6

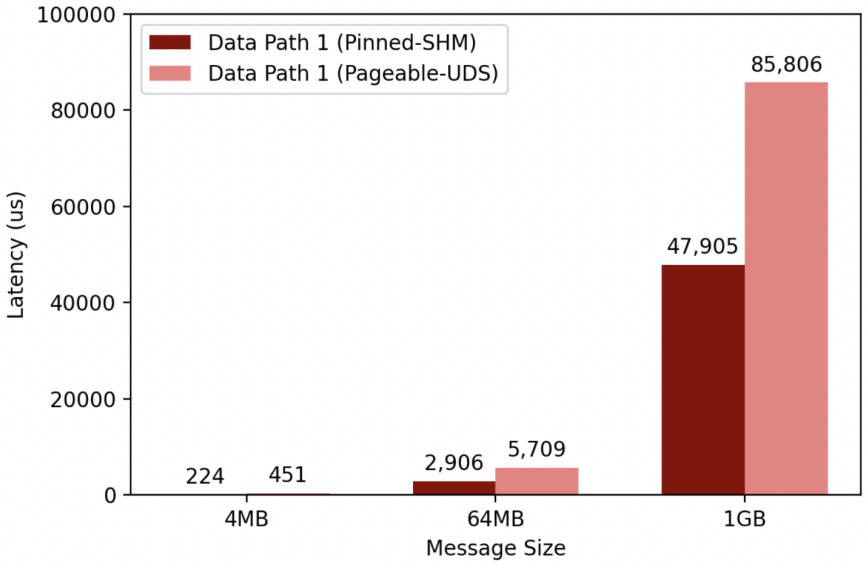


Figure 11: Device-to-host memory transfer latency of Pinned-SHM and Pageable-UDS in Data Path 1.

proposed Pinned-SHM architecture with the baseline Pageable-UDS approach.

Table 2 reveals the highly optimized profile of Pinned-SHM. The complexities of TLSF memory allocation and signaling are effectively minimized to a constant negligible overhead ($\approx 1\mu$s). Consequently, the latency of this data path is almost exclusively dominated by the *cudaMemcpy* operation.

In contrast, Table 3 exposes the severe inefficiency of copying large payloads through kernel socket buffers in the baseline. The Unix Domain Socket transmission introduces massive, linearly growing latency. Ultimately, this comparison highlights a critical architectural shift: by

Table 2: Latency breakdown of Data Path 1 (Pinned-SHM).

Message Size	T1: TLSF Malloc	T2: cudaMemcpy	T3: Notification via shared memory
4 MB	1	224	1
64 MB	1	2,906	1
1 GB	1	47,905	1

Unit: μs

Table 3: Latency breakdown of Data Path 1 (Pageable-UDS).

Message Size	T1: cudaMemcpy	T2: IPC (UDS)
4 MB	451	1,184
64 MB	5,709	17,810
1 GB	85,806	208,318

Unit: μs

transitioning from a software-bound mechanism (heavy IPC) to a hardware-bound one (pinned memory and direct DMA).

5.3 Data Path 2 Results

Data Path 2 measures cross-node transport. RGAPS uses RDMA Write with Immediate into the receive-side ring buffer, whereas the baseline uses TCP/IP. As shown in Figure 12, for a 4 MB message, RGAPS achieved 305 μs compared with 3,365 μs for TCP/IP. For 64 MB, the latency was 4,620 μs versus 39,832 μs. For 1 GB, the latency was 73,115 μs versus 602,994 μs.

The relative advantage is already large for small messages and becomes even more significant as payload size grows, showing that the software overhead of TCP/IP strongly dominates a CPU-centric transport path at small message sizes.

5.4 Data Path 3 Results

Data Path 3 measures local fan-out on the subscriber machine. RGAPS copies each message exactly once into shared GPU memory and then distributes only metadata. The baseline distributes a host-resident payload and forces each subscriber to perform its own *cudaMemcpy* into private GPU memory.

Table 4 demonstrates the superior scalability of our proposed method. By depositing data into a shared GPU memory segment via a single *cudaMemcpy*, the heavy host-to-device transfer cost becomes constant. With negligible notification overhead, the total latency remains stable regardless of the subscriber count (N).

In stark contrast, Table 5 reveals that the baseline approach scales poorly, suffering from a linear increase in total latency due to repeating the *cudaMemcpy* for each of the N subscribers.

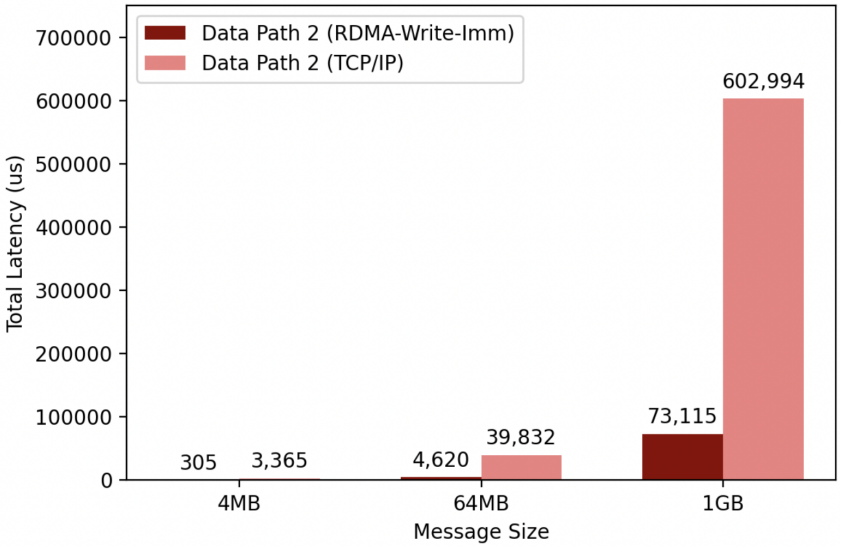


Figure 12: Total latency of Data Path 2 (RDMA-Write-Imm) and Data Path 2 (TCP/IP).

Table 4: Latency breakdown of Data Path 3 (Pinned-Shared).

Message Size	**T1:**	**T2:**	**T3: Notification via Iceoryx (Avg Latency)**			
	TLSF Malloc	**cudaMemcpy**	**N = 1**	**N = 2**	**N = 4**	**N = 8**
4 MB	1	216	6	7	9	11
64 MB	1	2,872	6	7	9	11
1 GB	1	43,677	6	7	9	11

* N: Number of subscribers Unit: μs

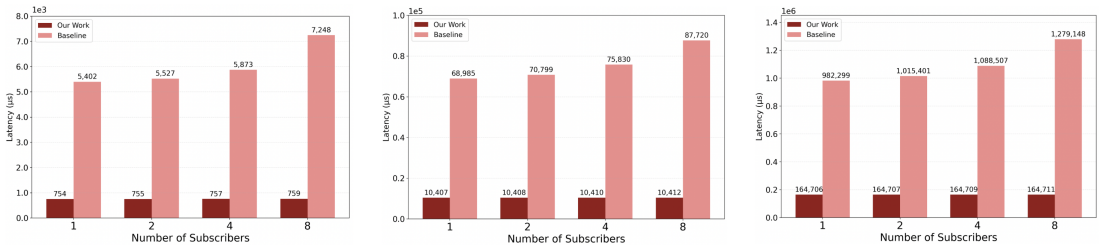
Table 5: Latency breakdown of Data Path 3 (Pageable-Iceoryx).

Message Size	**T1: Publish message via Iceoryx**				**T2: cudaMemcpy (Avg Latency)**			
	N=1	**N=2**	**N=4**	**N=8**	**N=1**	**N=2**	**N=4**	**N=8**
4 MB	6	7	7	10	396	520	866	2,238
64 MB	9	9	11	12	5,625	7,439	12,468	24,357
1 GB	9	10	11	12	85,172	118,273	191,378	382,018

* N: Number of subscribers Unit: μs

5.5 End-to-End Results

The most important metric is end-to-end latency from the moment a publisher issues a publish operation on the source machine until a subscriber on the destination machine can access a GPU-resident payload. Figure 13 reports this comparison across message sizes and fan-out levels. For 4 MB messages, RGAPS stayed near 754–759 μs across one to eight subscribers, while the baseline ranged from 5,402 to 7,248 μs. For 64 MB messages, RGAPS stayed near 10,407–10,412 μs, while the baseline rose from 68,985 to 87,720 μs. For 1 GB messages, RGAPS remained near 164,706–164,711 μs, whereas the baseline ranged from 982,299 to 1,279,148 μs.



(a) 4 MB (b) 64 MB (c) 1 GB

Figure 13: Total latency comparison across different message sizes.

Overall, the end-to-end improvement is roughly 6×–10×, and the largest gains appear when fan-out is high. The reason is straightforward: RGAPS keeps the dominant local materialization cost constant per message, while the baseline adds more and more duplicated host-to-device work as subscribers are added.

6 Discussion and Limitations

RGAPS demonstrates that a distributed GPU-aware data plane can substantially outperform CPU-centric pub/sub communication when payloads are large and reused by multiple GPU consumers. The main reason is not merely the use of RDMA in isolation; it is the integration of three ideas into one end-to-end pipeline: GPU-accessible pinned-memory staging on the sender, one-sided CPU-bypassing network transport, and one-time materialization into shared GPU memory on the receiver.

The framework is well suited to pipelines in which one GPU stage produces large intermediate artifacts for several downstream consumers. Example scenarios include distributed inference pipelines, real-time vision systems, multi-stage analytics on image or tensor streams, and other GPU-driven processing graphs in which the same payload must be reused rather than independently replicated.

RGAPS also has several limitations. First, the current platform does not support GPUDirect RDMA [6], so the design must still stage network I/O through host DRAM. Direct RNIC-to-GPU transfers would likely reduce latency further. Second, the current study focuses on a two-host deployment; multi-hop and larger multi-host systems will require additional work on routing and resource management. Third, the present implementation uses a single global receive ring buffer, which can become a scalability bottleneck under very high load. A natural next step is to allocate per-topic ring buffers with parallel transfer threads. Finally, RGAPS currently

provides live-only delivery semantics and does not yet support bounded replay or latest-value delivery for late joiners.

7 Conclusion

This paper presented RGAPS, a low-latency RDMA-based publish/subscribe framework for inter-machine GPU communications. RGAPS extends single-host GPU-aware pub/sub to a distributed two-host environment by combining shared CUDA-RDMA pinned-memory aggregation, RDMA Write with Immediate transport, and shared-GPU local fan-out on the receiver.

Experiments on a real GPU-RDMA platform showed that the proposed design reduces latency in each major stage of the communication path and improves end-to-end delivery by roughly 6×–10× relative to CPU-centric baselines. Just as importantly, RGAPS keeps local fan-out cost nearly constant as the number of subscribers increases, because each message is materialized into GPU memory only once.

These results indicate that low-latency inter-machine GPU pub/sub can be achieved without direct GPUDirect RDMA support, provided that host-side staging, network transport, and receive-side fan-out are designed as a unified data plane. RGAPS therefore offers a practical foundation for distributed GPU applications that require high-bandwidth, low-latency dissemination of large payloads across machines.

8 Acknowledgments

This research was supported by the National Science and Technology Council (NSTC), Taiwan, under Grants 112-2221-E-002-159-MY3, 115-2119-M-002-006-, and 113-2634-F-002-001-MBK.

References

[1] Aleksandar Dragojević, Dushyanth Narayanan, Orion Hodson, and Miguel Castro. FaRM: Fast Remote Memory. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*, Seattle, WA, 2014. USENIX Association.

[2] Xinhao Kong, Jingrong Chen, Wei Bai, Yechen Xu, Mahmoud Elhaddad, Shachar Raindel, Jitendra Padhye, Alvin R. Lebeck, and Danyang Zhuo. Understanding RDMA Microarchitecture Resources for Performance Isolation. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, Boston, MA, 2023. USENIX Association.

[3] Hao-En Kuan, Yung-Hsiang Yang, Zen-Mou Jiang, Chi-Sheng Shih, and Shih-Hao Hung. Towards Low-Latency GPU-Aware Pub/Sub Communication for Real-Time Edge Computing. In *2025 IEEE 31st International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*. IEEE, 2025.

[4] Linux man-pages project. ibv_reg_mr(3) — Linux manual page, 2024. https://man7.org/linux/man-pages/man3/ibv_reg_mr.3.html.

[5] Miguel Masmano, Ismael Ripoll, Alfons Crespo, and Jorge Real. TLSF: a New Dynamic Memory Allocator for Real-Time Systems. In *Proceedings. 16th Euromicro Conference on Real-Time Systems, 2004. ECRTS 2004.*, pages 79–88. IEEE, 2004.

[6] NVIDIA Corporation. *GPUDirect RDMA*, 2024. https://docs.nvidia.com/cuda/gpudirect-rdma/.

[7] NVIDIA Corporation. *RDMA Aware Networks Programming User Manual: Transport Modes*, 2024. https://docs.nvidia.com/networking/display/rdmaawareprogrammingv17/transport+modes.

[8] NVIDIA Corporation. CUDA Runtime API: 6.10. Memory Management, 2026. https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY.html.

[9] NVIDIA Corporation. How to Optimize Data Transfers in CUDA C/C++, 2026. https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/.

[10] sysprog21. tlsf-bsd: Two-Level Segregated Fit memory allocator implementation, 2024. https://github.com/sysprog21/tlsf-bsd.

[11] Konstantin Taranov, Steve Byan, Virendra Marathe, and Torsten Hoefler. KafkaDirect: Zero-copy Data Access for Apache Kafka over RDMA Networks. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*, pages 2191–2204. ACM, 2022.

[12] The Apache Software Foundation. Apache Kafka, 2024. https://kafka.apache.org/.

[13] The Eclipse Foundation. Eclipse iceoryx: True Zero-Copy Inter-Process-Communication, 2024. https://iceoryx.io/.

[14] The Eclipse Foundation. Eclipse Zenoh: Zero Overhead Pub/Sub, Store/Query and Compute, 2024. https://zenoh.io/.

[15] The Eclipse iceoryx Project. eclipse-iceoryx/iceoryx: True Zero-Copy Inter-Process-Communication Source Code, 2024. https://github.com/eclipse-iceoryx/iceoryx.

[16] The Eclipse Zenoh Project. eclipse-zenoh/zenoh: Blue Mountain (Rust) Implementation Source Code, 2024. https://github.com/eclipse-zenoh/zenoh.